

ExoHabitAI – Project Deployment Theory

Objective

This document explains the theoretical and practical deployment process of the ExoHabitAI application. The goal is to make the Machine Learning based habitability prediction system publicly accessible through a live web URL.

The deployment integrates:

- Trained Machine Learning Model
- Flask Backend API
- Frontend User Interface
- Production Web Server (Gunicorn)
- Cloud Hosting Platform (Render)

Deployment Overview

The deployed system follows a client-server architecture:

User → Frontend → Flask API → ML Model → Response → Display Results

The frontend collects planetary parameters from the user and sends them to the Flask backend. The backend loads the trained ML model, processes the input, and returns habitability predictions in JSON format. The frontend dynamically displays the results.

Pre-Deployment Checklist

Before deployment, the following conditions were verified:

- Final trained model saved inside models/ directory
- Flask backend runs locally without errors
- Frontend successfully communicates with backend
- requirements.txt updated with all dependencies
- All files committed and pushed to GitHub
- Correct project structure maintained

Backend Preparation for Deployment

The backend (app.py) includes:

- Flask application initialization
- Model loading using joblib
- /predict and /rank API routes
- JSON response handling

Production configuration includes:

```
if __name__ == '__main__':  
    app.run(debug=False)
```

Debug mode is disabled for production security.

requirements.txt Configuration

The following libraries were included in requirements.txt:

numpy
pandas
matplotlib
seaborn
scikit-learn
xgboost
flask
joblib
requests
plotly
gunicorn

Gunicorn acts as the production WSGI server.

Procfile Configuration

Procfile content:

```
web: gunicorn backend.app:app
```

This ensures the Flask app runs using Gunicorn in production.

Deployment on Render

Deployment Steps:

1. Created account on Render using GitHub login
2. Connected repository branch:
<https://github.com/GKSJ19/ExoHabitAI/tree/Prachi>
3. Selected Python environment
4. Build Command: `pip install -r requirements.txt`
5. Start Command: `gunicorn backend.app:app`
6. Service deployed successfully

Live Application URL:

<https://exohabitai-yb2d.onrender.com>

Testing the Deployed Application

After deployment, the following were tested:

- Homepage loads correctly
- /predict endpoint functions properly
- User inputs return accurate predictions
- No runtime errors or crashes
- Model loads successfully in cloud environment

Common Deployment Errors & Fixes

Error: Module Not Found

→ Ensure all dependencies are listed in requirements.txt

Error: Application Failed to Start

→ Verify Procfile syntax

→ Confirm correct Flask app path

Error: Model File Not Found

→ Ensure model .pkl file is committed to GitHub repository

Final Deliverables (Deployment Phase)

- Live deployment URL
- GitHub repository link
- Screenshot of working web application
- Confirmation of functional APIs
- Deployment documentation

Conclusion

The ExoHabitAI system has been successfully deployed on Render cloud platform. The trained Machine Learning model, Flask backend, and frontend interface are fully integrated and publicly accessible via the live URL.